Workflow for Preparing DHSVM Input Files Using QGIS–GRASS and a Pure Python Pipeline

(AR Watershed Example)

Yiyun Song

April 10, 2026

1 Overview

This document describes an end-to-end, pure Python workflow for preparing all required spatial and state input files for DHSVM for the AR watershed. The updated pipeline replaces legacy intermediate ASCII exports and C-engine utilities with direct binary serialization. It combines:

- QGIS with GRASS GIS for fundamental spatial preprocessing (DEM clipping, flow accumulation, stream extraction).
- A unified Python master script (`prep_dhsvm_inputs.py`) that coordinates topological routing, binary grid generation, and physical initial state computation.

All paths in this document use the example project directory:

`/Users/benthosyy/Desktop/CreateStreamNetwork_PythonV/DEM_AR_0406`

2 Directory Structure and Prerequisites

The workflow enforces strict directory routing to separate intermediate diagnostic artifacts from production-ready model inputs. This keeps the workspace organized and prevents accidental file overwrites during calibration.

Source directories (user-maintained):

- `Reprojected_DEM/` – Reprojected master DEMs.
- `Reprojected_Watersheds/` – Watershed polygon(s) in the working CRS.
- `scripts/` – Custom Python modules (detailed in Section 3):
  - `prep_dhsvm_inputs.py` (Master orchestrator)
  - `rowcolmap.py`, `roadaspect.py` (Row/Col indexing & stream aspect)

- `channelclass.py` (Channel classification)
- `soildepthscript.py` (Topographic soil depth)
- `dem_to_dhsvm_bins.py` (Base map serialization)
- `generate_dhsvm_states.py` (Initial state generator)
- `plot_mainstem.py` (Geomorphic validation)

Automatically generated output directories:

- `DHSVM_input_binaries/` – Flat binary grids (DEM, mask, soil, veg, depths).
- `DHSVM_input_streams/` – Topological files (map, network, class).
- `modelstate/` – Pure Python generated initial condition grids and channel states.
- `Intermediate_GIS/` – Diagnostic rasters and shapefiles for QA.

3 Pipeline Architecture and Module Functionality

The pipeline uses a modular Python architecture. Each script addresses a specific physical or computational requirement of DHSVM, so that individual stages can be modified or swapped without destabilizing the rest of the workflow.

3.1 Core Python Modules

- `prep_dhsvm_inputs.py` (Master Script): Orchestrates the entire workflow within the QGIS environment. It sequences the GRASS processing and dynamically imports all sub-modules through `importlib`, allowing each stage to be updated independently.
- `rowcolmap.py` & `roadaspect.py`: Essential geometric utilities. They map geographic coordinates to DHSVM's row/column matrix indices (with a top-left origin consistent with DHSVM map files) and compute local stream aspect to align surface runoff partitioning. `rowcolmap.py` also assigns deterministic `SegID` values to stream segments and performs a uniqueness sanity check.
- `channelclass.py`: Classifies vector stream segments based on topographic slope and upstream contributing area, generating the structured `stream.class.dat` routing table. Slope units (degrees, percent, or tan) are auto-detected from the attribute distribution and internally converted to $\tan(\theta)$ before binning, which makes the module portable across different GRASS output conventions.
- `soildepthscript.py`: Replaces constant depth assumptions by generating a topographically weighted soil depth raster using the PNNL weighted formula (slope, source area, and elevation terms). Parameters are calibrated to the Southern Appalachians / Coweeta Hydrologic Laboratory regime (`MIN_DEPTH` = 2.0 m, `MAX_DEPTH` = 6.0 m). It enforces the DEM mask as an absolute master mask so that boundary pixels lost by upstream 3×3 raster operations are restored, guaranteeing perfect pixel alignment for the DHSVM C-engine.

- `dem_to_dhsvm_bins.py`: Reads the foundational DEM and writes DHSVM-ready Float32 and Int8 base map binaries (`dem.bin`, `mask.bin`, `soil.bin`, `veg.bin`) directly via NumPy. It simultaneously generates a suite of uniform soil depth baselines (2.0 m through 4.0 m) that share the exact same spatial mask as the dynamic depth raster, enabling strictly pixel-aligned sensitivity experiments.

- `generate_dhsvm_states.py`: Initializes the cold-start conditions for the model. It writes Float32 grid states directly as flat binaries (with `.bin` explicitly appended to satisfy DHSVM's internal `ReadModelState` path concatenation), and dynamically parses `stream.class.dat` to compute physically accurate initial water volumes for each channel segment ($V = W \cdot L \cdot d_0$).

- `plot_mainstem.py` (Validation): A standalone analytical tool that traces the true physical main stem upstream from the outlet by enforcing elevation-based routing on the extracted stream network. It identifies geomorphic knickpoints via smoothed slope thresholds and exports longitudinal profile data and figures for quality assurance.

3.2 Rationale for a Pure Python Architecture

The transition from legacy methods (ASCII exports, `myconvert`, `MakeModelStateBin`, `MakeChannelState.sh`) to a pure Python pipeline resolves several long-standing computational and physical inconsistencies:

1. **Direct Binary Serialization:** NumPy's `tofile()` translates raster arrays directly into memory-contiguous Float32 and Int8 binaries. This eliminates the floating-point precision truncation inherent in ASCII text conversion, bypasses the OS-dependent `myconvert` utility, and reduces I/O execution time by roughly an order of magnitude.

2. **Dynamic Channel Width Parsing:** The legacy bash script (`MakeChannelState.sh`) hardcoded channel widths for a limited number of stream classes. For higher-order classes (e.g., Class 13), this resulted in unassigned widths, yielding initial channel volumes of zero and subsequent numerical shocks during the model spin-up phase. `generate_dhsvm_states.py` resolves this by dynamically parsing `stream.class.dat`, guaranteeing physical conservation of mass for any classification scheme.

3. **Circumventing C-Engine Memory Limitations:** Legacy utilities like `MakeModelStateBin` possess a hardcoded `NAMESIZE` buffer limit (typically 128 characters). Deeply nested project directories cause path concatenations to exceed this limit, triggering buffer overflows (Segmentation Faults / `SIGFPE` / trace traps). By writing the binary arrays natively in Python and explicitly appending the `.bin` extension, the pipeline conforms to DHSVM's `ReadModelState` I/O requirements without invoking the vulnerable C memory constraints.

4. **Endianness Compatibility:** The Python pipeline natively generates little-endian binaries that are bit-for-bit compatible with the DHSVM `ReadModelState` C-routines on the target platforms, removing the need for any platform-specific byte-swap step.

4 Single-Step Pipeline Execution

All spatial preprocessing, binary generation, and state initialization are executed via a single call to the master script in the QGIS Python Console.

Listing 1: Run `prep_dhsvm_inputs.py` in the QGIS Python Console.

```
exec(compile(Path('/Users/benthosyy/Desktop/CreateStreamNetwork_PythonV/DEM_AR_0406/scripts/prep_dhsvm_inputs.py').read_text(),
    '/Users/benthosyy/Desktop/CreateStreamNetwork_PythonV/DEM_AR_0406/scripts/prep_dhsvm_inputs.py', 'exec'))
```

4.1 Execution Stages and Console Output

The master script orchestrates four primary stages, reporting progress directly to the console:

1. **Spatial Initialization & GRASS Processing.** Clips the DEM to the watershed polygon, computes flow direction and flow accumulation (`r.watershed`), extracts the stream network (`r.stream.extract`), and derives topographic slope (`r.slope.aspect`). All intermediate artifacts (e.g., `elev_clipped.tif`, `flow_acc.tif`, `flow_dir.tif`, `stream_raster.tif`, `stream_slope.tif`, `streamfile.shp`) are routed to `Intermediate_GIS/`.

2. **Stream Routing & Classification.** Calls `rowcolmap.py` and `roadaspect.py` to assign row/column indices, `SegID`s, and local aspect to every segment, then `channelclass.py` to classify segments by slope band and contributing area. The topological files `stream.map.dat`, `stream.network.dat`, and `stream.class.dat` are written directly to `DHSVM_input_streams/`.

3. **Soil Depth & Base Map Serialization.** Calls `soildepthscript.py` to compute the dynamic topographic soil depth raster (`soildepth.bin` and `soildepth.tif` for diagnostics), then `dem_to_dhsvm_bins.py` to serialize the master DEM, mask, soil, and vegetation layers directly into Float32/Int8 flat binaries. A suite of uniform soil depth baselines (`soildepth_uniform_2.0m.bin` through `4.0m.bin`) is generated alongside them to serve as control groups for subsurface stormflow sensitivity experiments. All binaries are saved to `DHSVM_input_binaries/`.

4. **Initial State Generation.** Calls `generate_dhsvm_states.py`. This pure Python module bypasses the legacy `MakeModelStateBin` C-utility. It writes Float32 state grids for interception, snow, and soil (with `.bin` explicitly appended), and parses the actively generated `stream.class.dat` to compute physically accurate initial water volumes in the channel state text file. All state outputs are routed to `modelstate/`.

The console output confirms the successful generation and precise routing of all spatial and state files. *(The complete, step-by-step QGIS Python console execution log is provided in Appendix A.)*

5 Summary of Prepared DHSVM Inputs

After executing the master script, the AR watershed workspace contains a complete set of DHSVM input files partitioned into strict directories:

- `DHSVM_input_binaries/` (Gridded spatial inputs):
  - `dem.bin`, `mask.bin`, `soil.bin`, `veg.bin`
  - `soildepth.bin` (dynamic topographic depth)
  - `soildepth_uniform_2.0m.bin` ... `soildepth_uniform_4.0m.bin` (sensitivity baselines)
- `DHSVM_input_streams/` (Topological routing):
  - `stream.map.dat`, `stream.network.dat`, `stream.class.dat`
- `modelstate/` (Initial conditions at `01/01/2016-00`):
  - `Interception.State.01.01.2016.00.00.00.bin` (Float32 binary)
  - `Snow.State.01.01.2016.00.00.00.bin` (Float32 binary)
  - `Soil.State.01.01.2016.00.00.00.bin` (Float32 binary)
  - `Channel.State.01.01.2016.00.00.00` (ASCII text format)

These directories can be directly referenced in the `.dhs` configuration file to initiate a DHSVM simulation cleanly.

6 Workflow Summary Diagram

Figure 1 illustrates the streamlined architecture of the pure Python pipeline, including the optional geomorphic validation branch.

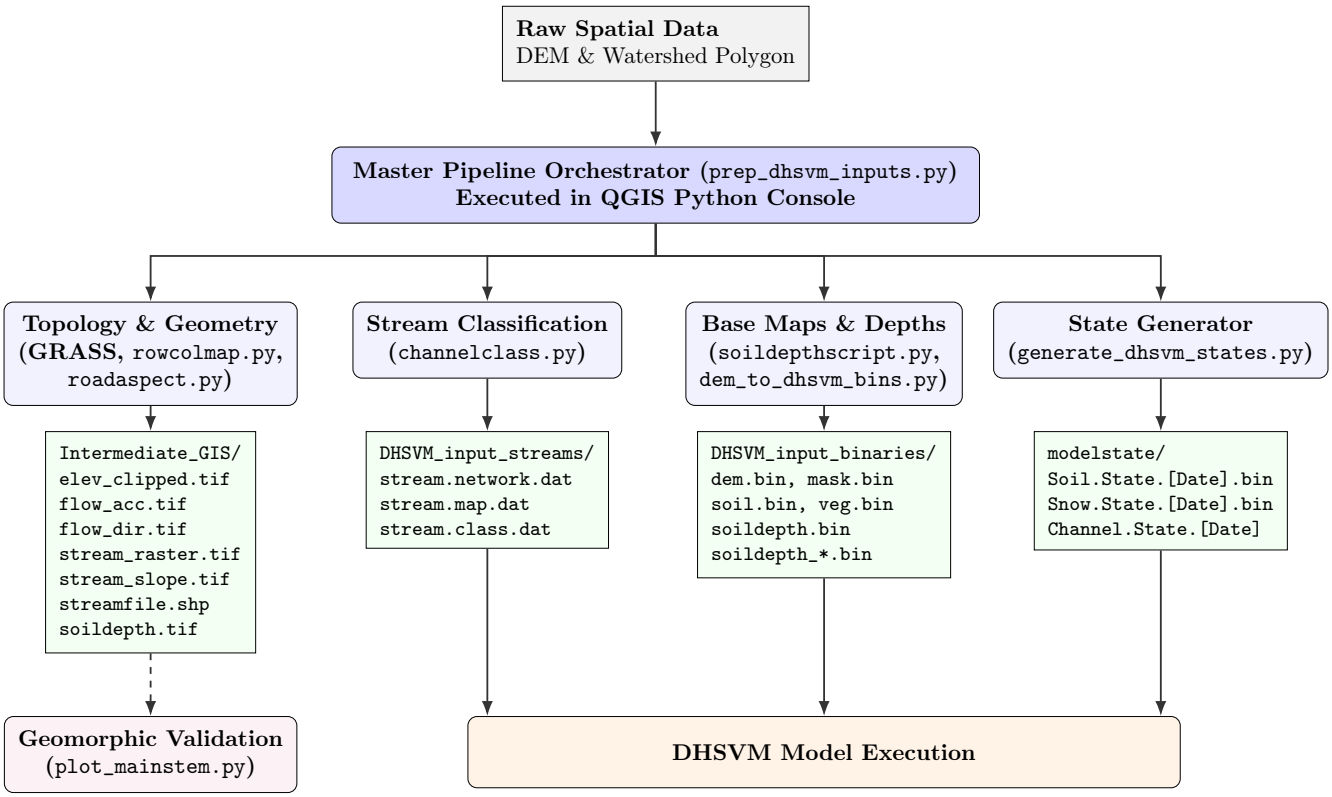

Figure 1: Architecture of the pure Python DHSVM spatial preparation workflow. Solid arrows indicate the production pipeline; the dashed arrow indicates the optional geomorphic QA branch driven by `plot_mainstem.py`.

A Full QGIS Console Execution Log

The following log demonstrates the successful execution of the single-step master pipeline (`prep_dhsvm_inputs.py`) for the AR watershed, outputting directly to the strict directory architecture.

Listing 2: Full console log of the DHSVM spatial pipeline execution.

```
# Python Console
# Use iface to access QGIS API interface or type '?' for more info
# Security warning: typing commands from an untrusted source can harm your computer
exec(compile(Path('/Users/benthosyy/Desktop/CreateStreamNetwork_PythonV/DEM_AR_0406/scripts/prep_dhsvm_inputs.py').read_text(),
    '/Users/benthosyy/Desktop/CreateStreamNetwork_PythonV/DEM_AR_0406/scripts/prep_dhsvm_inputs.py', 'exec'))

=======================================================
  DHSVM SPATIAL PIPELINE INITIALIZING
=======================================================
[ok] Watershed polygon: arwd_watershed_UTM17.shp
[ok] Master DEM: elev_clipped.tif
[info] Cell area ≈ 792.858 m²
[info] MIN_SRC_AREA (m²) = 47571.500  (from 60 cells)
[step] Clipping DEM to watershed...
[ok] Working DEM ready: elev_clipped.tif
[step] GRASS r.watershed -> flow direction & accumulation...
[step] GRASS r.stream.extract -> extracting streams...
[step] GRASS r.slope.aspect -> calculating slope...
[step] Routing classifications to DHSVM_input_streams/...
stream.class.dat written:
    /Users/benthosyy/Desktop/CreateStreamNetwork_PythonV/DEM_AR_0406/DHSVM_input_streams/stream.class.dat
Class counts: {13: 30}
[info] Slope mode auto-detected as: tan (converted to tan for binning)
[step] Writing topologic stream files to DHSVM_input_streams/...
[step] Compute Soil Depth Binary (soildepthscript.py)...
  [soildepth] Reading spatial inputs into memory...
  [soildepth] Aligning boundaries and sanitizing edge pixels...
  [soildepth] Computing weighted topograpic functions...
  [soildepth] Writing flat binary -> soildepth.bin
  [soildepth] Writing GeoTIFF -> soildepth.tif
  [soildepth] Complete.
[step] Generate DHSVM Base Maps (dem_to_dhsvm_bins.py)...

--- Initializing DHSVM Base Map Generation Pipeline ---
[step] Reading master topography: elev_clipped.tif
[step] Structuring physical memory arrays (Float32 and Int8)...
[step] Serializing standard base maps to: DHSVM_input_binaries/
  -> Generated: dem.bin  (Float32)
  -> Generated: mask.bin (Int8)
  -> Generated: soil.bin (Int8)
  -> Generated: veg.bin  (Int8)

[step] Generating uniform soil depth baselines...
  -> Generated: soildepth_uniform_2.0m.bin (Float32)
  -> Generated: soildepth_uniform_2.5m.bin (Float32)
  -> Generated: soildepth_uniform_3.0m.bin (Float32)
  -> Generated: soildepth_uniform_3.5m.bin (Float32)
  -> Generated: soildepth_uniform_4.0m.bin (Float32)

[SUCCESS] DHSVM spatial base maps and depth baselines successfully generated!
[step] Generate DHSVM Initial States (generate_dhsvm_states.py)...
```

```
=======================================================
  DHSVM PURE PYTHON STATE INITIALIZER
=======================================================

[step] Generating binary grid states for 01.01.2016.00.00.00 (NY=55, NX=72)
  -> Interception.State.01.01.2016.00.00.00.bin (pure Python binary)
  -> Snow.State.01.01.2016.00.00.00.bin (pure Python binary)
  -> Soil.State.01.01.2016.00.00.00.bin (pure Python binary)
[step] Generating physics-based channel states -> Channel.State.01.01.2016.00.00.00
=======================================================



=======================================================
  PIPELINE FINISHED SUCCESSFULLY
=======================================================
STREAM FILES:  DHSVM_input_streams/
BINARY GRIDS:  DHSVM_input_binaries/
MODEL STATES:  modelstate/
INTERMEDIATE:  Intermediate_GIS/
```